Spark Plug Examples
Reference

Fonteva Spark Plugs extend Fonteva's functionality by enabling the injection of custom Lightning Components into default Fonteva process flows. This article contains example code and outlines as a guide to implementing Fonteva Spark Plug functionality.

Notes

- **Custom Lightning Component** - The component requires a specific interface to ensure all data from the Spark Plug is pushed to the component. The component must be global, and implement `FDService:SparkPlugComponentInterface`.
  - `FDService:SparkPlugLoadedEvent` - This event is executed when the Lightning Component is fully loaded to hide the loader.
  - `FDService:SparkPlugCompleteEvent` - This event is executed when the Lightning Component's requirements are satisfied. This instructs the container to show the next component, or hide the container if no components are left to display.

    > **WARNING**: Do not use `FDService:SparkPlugCompleteEvent` with the **Process Refund** or **Tax and Shipping** Spark Plug Extension Points.

- **Custom Metadata Type record** - The configuration record that instructs the container to display the configured Lightning Component.

Example: Add to Cart

This example outlines how to display a custom Lightning Component when the **Add to Cart** button is selected on the **Item Detail** page.

SparkPlugExample.cmp

Implements the `FDService:SparkPlugComponentInterface` and has global access. The `init` method sets up an initialization handler that calls the `doInit` method in the component's controller when the component is initialized. The component displays "Demo Spark Plug" when the "Add to cart" button is selected for a Store Item.

```
<aura:component description="SparkPlugExample" implements="FDService:SparkPlugComponentInterface" access="gl
      <aura:handler name="init" value="{!this}" action="{!c.doInit}"/>
      <aura:registerEvent type="FDService:SparkPlugCompleteEvent" name="SparkPlugCompleteEvent"/>
      <aura:registerEvent type="FDService:SparkPlugLoadedEvent" name="SparkPlugLoadedEvent"/>
       Demo Spark Plug!!!!
      <lightning:button variant="brand" label="Submit" onclick="{! c.handleClick }" />
</aura:component>
```

Home (/s/)

SparkPlugExample.cmp-meta.xml

The XML file that defines the metadata and used to define and describe the Aura component bundle within the Salesforce platform, specifying its API version and giving a brief description for easier identification and management.

```
<?xml version="1.0" encoding="UTF-8"?>
    <AuraDefinitionBundle xmlns="http://soap.sforce.com/2006/04/metadata">
         <apiVersion>45.0</apiVersion>
         <description>SparkPlugExample</description>
    </AuraDefinitionBundle>
```

SparkPlugExampleController.js

The JavaScript controller for the Aura component which defines the behavior for two key actions: initialization and button click. The controller manages the component's lifecycle and user interactions by firing specific events during initialization and, on button click, allowing other parts of the application to respond accordingly.

```
    ({
    doInit : function(component, event, helper) {
        var compEvent = $A.get('e.FDService:SparkPlugLoadedEvent');
        compEvent.setParams({extensionPoint : component.get('v.extensionPoint')});
        compEvent.fire();
    }
    ,
    handleClick : function(component, event, helper) {
        var compEvent = $A.get('e.FDService:SparkPlugCompleteEvent');
        compEvent.setParams({extensionPoint : component.get('v.extensionPoint')});
        compEvent.fire();
    }
})
```

Framework__Spark_Plug_Extension.Add_To_Cart_Example.md-meta.xml

Defines settings and configurations for the feature in a Salesforce application, specifying the associated Lightning Component, its order, and the extension point.

```
<?xml version="1.0" encoding="UTF-8"?>
    <CustomMetadata xmlns="http://soap.sforce.com/2006/04/metadata" xmlns:xsi="http://www.w3.org/2001/XMLSchem
    <label>Add To Cart Example</label>
    <protected>false</protected>
    <values>
        <field>Framework__Order__c</field>
        <value xsi:type="xsd:double">1.0</value>
    </values>
    <values>
        <field>Framework__Lightning_Component__c</field>
        <value xsi:type="xsd:string">SparkPlugExample</value>
    </values>
    <values>
        <field>Framework__Spark_Plug_Extension_Point__c</field>
        <value xsi:type="xsd:string">On_Click_Add_to_Order</value>
    </values>
  </CustomMetadata>
```

Example: Process Refund

This example outlines how to display a custom Lightning Component when the **Process Refund** button is selected on the **Refund** page.

SparkPlugExampleForProcessRefund.cmp

Implements the `FDService:SparkPlugComponentInterface` and has global access. The `init` method sets up an initialization handler that calls the `doInit` method in the component's controller when the component is initialized. The component displays "Demo Spark Plug" when the "Process Refund" button is selected on the Refund Receipt.

```
<aura:component description="SparkPlugExampleForProcessRefund" implements="FDService:SparkPlugComponentInterfa
        <aura:handler name="init" value="{!this}" action="{!c.doInit}"/>
        <aura:attribute name="data" type="Map" access="global"/>
        <aura:registerEvent type="FDService:SparkPlugLoadedEvent" name="SparkPlugLoadedEvent"/>

         Demo Spark Plug!!
    <div><lightning:button variant="brand" label="processRefund" onclick="{!c.processRefund2 }" /></div>
</aura:component>
```

SparkPlugExampleForProcessRefund.cmp-meta.xml

The XML file that defines the metadata and used to define and describe the Aura component bundle within the Salesforce platform - specifying its API version and giving a brief description for easier identification and management.

```
<?xml version="1.0" encoding="UTF-8"?>
<AuraDefinitionBundle xmlns="http://soap.sforce.com/2006/04/metadata">
    <apiVersion>59.0</apiVersion>
    <description>SparkPlugExampleForProcessRefund</description>
</AuraDefinitionBundle>
```

SparkPlugExampleForProcessRefundController.js

The JavaScript controller for the Aura component which defines the behavior for two key actions: initialization and button click. The controller manages the component's lifecycle and user interactions by firing specific events during initialization and, on button click, allowing other parts of the application to respond accordingly.

```
({
        doInit : function(component, event, helper) {
        let compEvent = $A.get('e.FDService:SparkPlugLoadedEvent');
        compEvent.setParams({extensionPoint : component.get('v.extensionPoint')});
        compEvent.fire();
        },
    processRefund2 : function(component, event, helper) {
        let action = component.get('c.processRefund');
        let recId = component.get('v.data').replaceAll('"','');
        action.setParams({
            receiptId: recId
        });
        action.setCallback(this, function(result) {
            if (result.getState() === 'SUCCESS') {
                window.location = '/' + recId;
            } else {
                console.log('failed');
            }
        });
        $A.enqueueAction(action);
    }
})
```

SparkPlugs/main/default/classes/MySparkPlugController.cls

The Apex controller class which includes a method to process refunds. The Apex controller method processes a refund for a given receipt ID. It provides two variants of the refund processing logic, one using the `FDService.RefundService` and another using a direct field update on the receipt record.

```
public class MySparkPlugController {

    @AuraEnabled
    public static void processRefund(String receiptId) {

        OrderApi__Receipt__c receipt = (OrderApi__Receipt__c)new Framework.Selector(OrderApi__Receipt__c.SObje
                        .selectById(receiptId);
        FDService.Receipt postedReceipt = ((FDService.IRefundService3)FDService.RefundService.getInstance()).p

        // 2nd variant to process refund - comment out lines above, uncomment out lines below
        // OrderApi__Receipt__c receipts = (OrderApi__Receipt__c) new Framework.Selector(OrderApi__Receipt__c.
        // receipts.OrderApi__Process_Refund__c = true;
        // Framework.SObjectService.updateRecords(receipts);
    }
}
```

Example: New Billing Address

In this example, create new Billing Addresses through a custom Spark Plug component and display them in the Billing Address section of the Checkout Page Payment section.

NOTE: This example applies to Fonteva release **21Winter.0.46**, or later.

Preparation

Register the `OrderApi:KnownAddressChangeEvent` within the **FetchShippingTaxAddress** component. Ensure this event is executed when the **Proceed to Billing** button is selected. The event parameters must include:

- **uniqueId**: Set to **SparkPlugKnownAddressUpdateEvent**
- **value**: This is an object containing **selectedAddressID**, which contains the **Salesforce ID** of the selected known address.

Example Event Parameters

```
{
  "uniqueId": "SparkPlugKnownAddressUpdateEvent",
  "value":
  {
       "selectedAddressId": "a1B2C3D4E5F6G7H8I9J0" // Replace with the actual Known Address ID
  }
}
```

Resources

- Fonteva Spark Plugs (https://fun.fonteva.com/s/article/fonteva-spark-plugs). Feature Overview
- Enable and Configure Spark Plugs (https://fun.fonteva.com/s/article/enable-and-configure-spark-plugs). How to Guide
- Spark Plug Extension Points (https://fun.fonteva.com/s/article/spark-plug-extension-points). Reference

Published: July 24, 2024
Article Number: 000002341

Suggested Articles